TD n°2: N-Lateration Positioning System

Positioning and Dynamic Mobility

Ahmed Almuharaq
Master 1 IoT
Université Marie et Louis Pasteur, UFR STGI
Montbéliard, France

March 7, 2026

Abstract

This report details the conception and implementation of a basic N-lateration algorithm for indoor positioning The objective is to estimate the position of a Mobile Terminal by minimizing the sum of distances to 4-spheres inside an aggregated perimeter The implementation utilizes a Factory design pattern for dataset initialization and compares a brute-force spatial walk algorithm with a geometric resolution approach

1 Introduction

N-lateration is a fundamental geometric technique used in both indoor and outdoor positioning systems to determine the location of a receiver based on the calculated distances to multiple emitters In a 3D space, determining a position requires at least 4 distances between emitters and a receiver

Distances are traditionally derived using either celerity models ($d = c \times t$) or signal attenuation models, such as the Friis propagation model

$$\frac{Pe}{Pa} = \frac{G_{oe} \times G_{ar} \times \lambda}{4\pi} \left(\frac{r}{d}\right)^2 \tag{1}$$

2 System Architecture

The software architecture was designed using object-oriented principles to ensure modularity and separation of concerns. The design follows the Factory pattern to initialize the provided testing dataset

2.1 Object Class Diagram

The architecture relies on abstractions (`ICell`, `IPosition`) rather than concrete implementations to maintain flexibility. The prioritized functions, including pre-processing, dataset generation, and the core spatial walk algorithms, are encapsulated within the `NLateration` main class

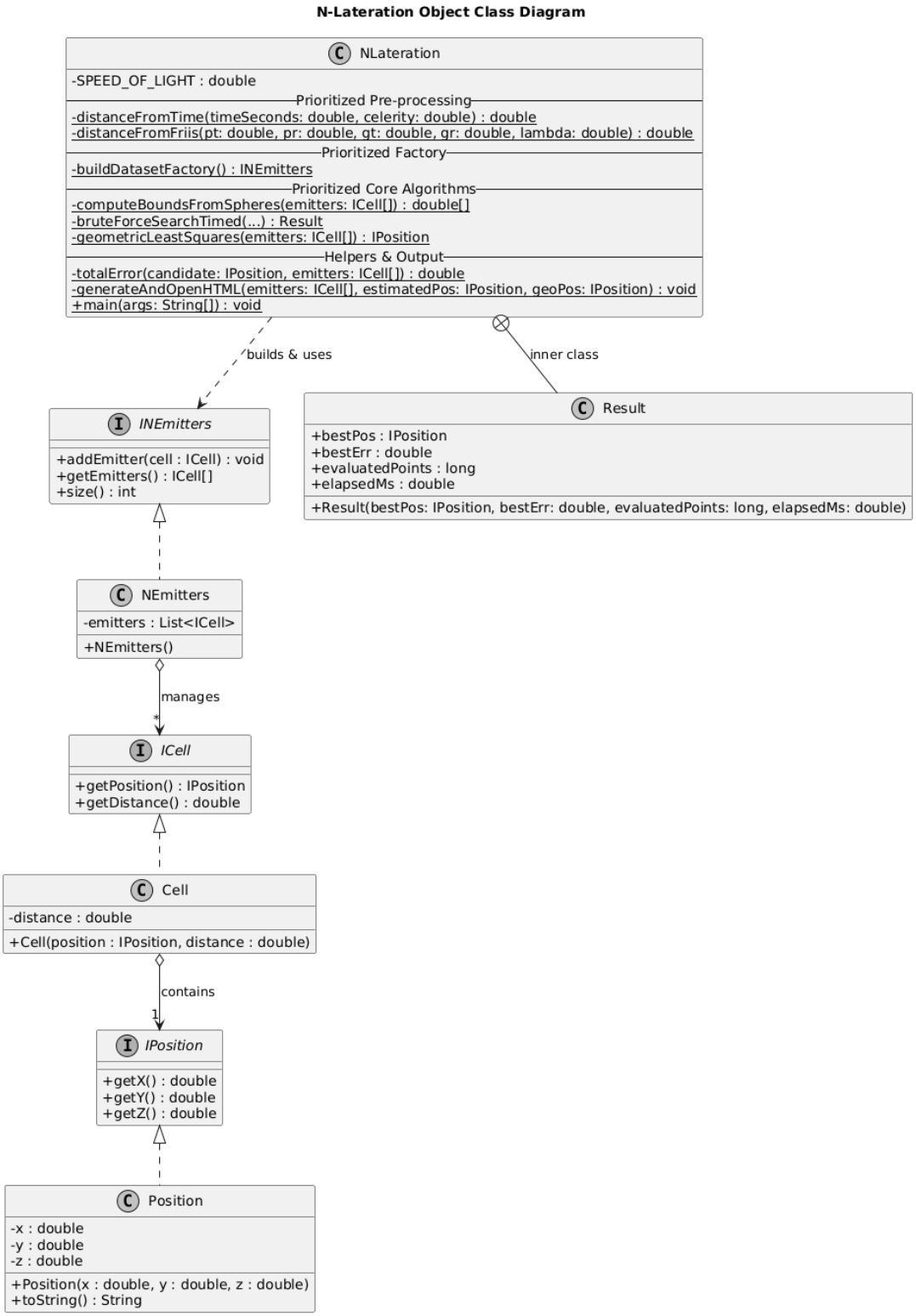

Figure 1: UML Class Diagram of the N-Lateration System.

3 Methodology

3.1 Dataset Initialization

A specific indoor environment scenario involving 4 Access Points ($N = 4$) and a mobile object was analyzed The coordinates and distances to the receiver are defined as follows:

- $E_0(0.5, 0.5, 0.5)$ with $d = 3.0$
- $E_1(4.0, 0.0, 0.0)$ with $d = 2.0$
- $E_2(4.0, 5.0, 5.0)$ with $d = 4.2$
- $E_3(3.0, 3.0, 3.0)$ with $d = 2.5$

3.2 Brute-Force Spatial Walk (V1)

The V1 algorithm iterates through a bounding box defined by the aggregated perimeters of the 4 spheric volumes It evaluates each point in space to find the coordinate that minimizes the sum of absolute errors ($L1$ norm) between the geometric distance and the measured distance to all spheres

3.3 Geometric Resolution Comparison

To validate the V1 implementation, the results were compared against a geometric resolution By subtracting the equation of the first sphere from the subsequent spheres, the quadratic terms are eliminated, resulting in a linear system of equations ($Ap = b$) that can be solved using standard matrix inversion to find the exact intersection point.

4 Results and Visualization

The implementation accurately identifies the spatial intersection. To properly verify the algorithm's output, a 3D visualization module was developed using HTML and Plotly.js.

The visualizer plots the 4 emitters alongside their semi-transparent distance spheres. Dashed lines connect the emitters to the computed intersection point, demonstrating how the minimized distances converge to estimate the receiver's location.

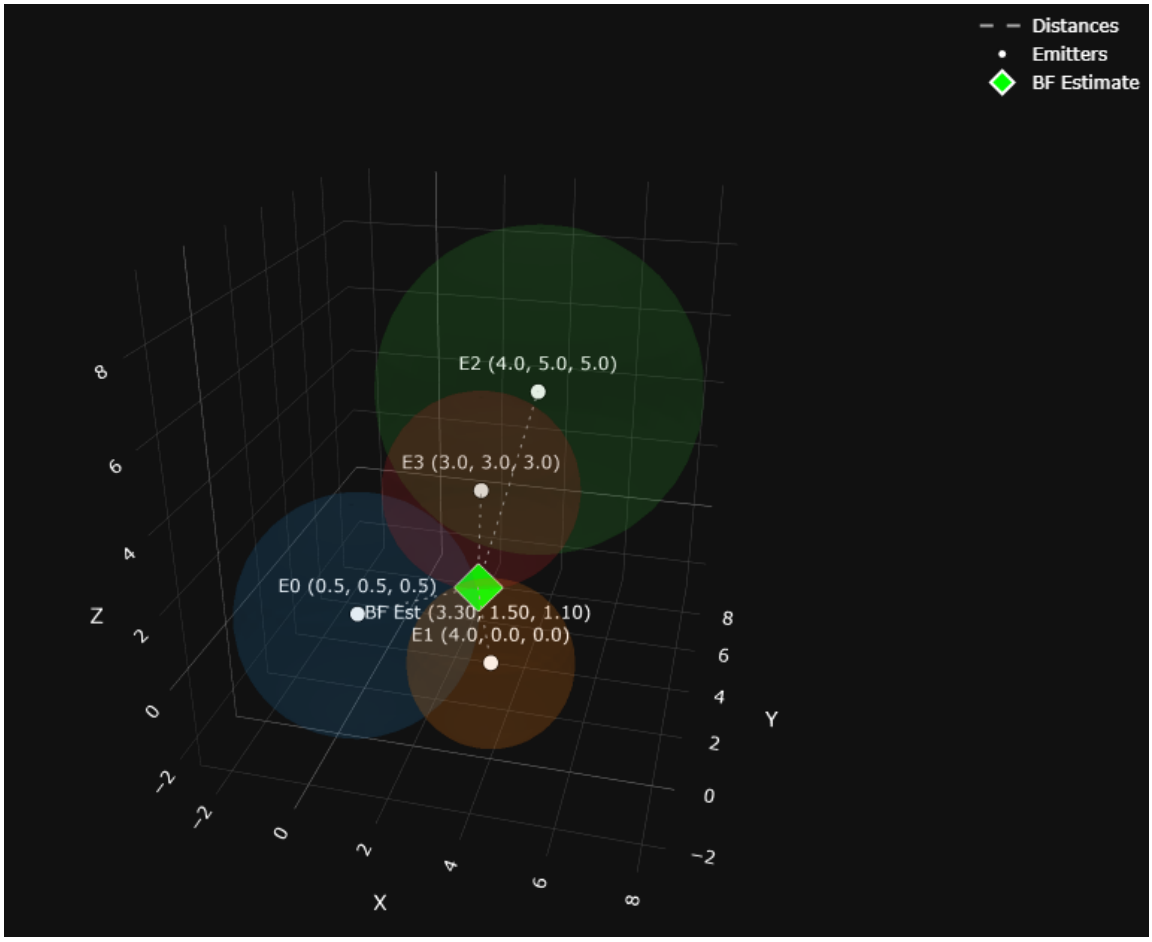

Figure 2: 3D visualization of the intersecting 4-spheres and the estimated position.

5 Conclusion

This directed work successfully implemented the N-lateration algorithm for 3D indoor positioning By combining robust object-oriented architecture with both iterative spatial search and geometric linear resolution, the system can reliably pinpoint mobile terminals provided accurate distance measurements.